ReSiE Security Hardening

Methodology summary of the changes from the original source archive to the final hardened version, including flexible local use, a dedicated confined server entry point, and support for additional JSON metadata.

19	1	+1,536 / -609	2
source files changed	central security module added	source-line diff	test harness files aligned

Core design decision

Security policy is selected by trusted caller code, not by the project JSON. Normal local execution remains flexible. Uploaded server jobs use a separate confined-only API or CLI command, and project data cannot downgrade that mode or choose the trusted roots.

Execution model

Local mode

- Default for the normal API and `run` command.
- Allows absolute paths, parent paths, symlinks, and unrelated input/output folders for trusted desktop use.
- Existing examples continue resolving relative paths from the ReSiE project root.
- Regular-file checks, bounded parsers, finite-number validation, allocation limits, graph limits, output encoding, and concurrency fixes remain active.

Confined server mode

- Exposed through `load_and_run_confined` and `run-confined`.
- Requires server-owned input and output roots and provides no local-mode switch.
- Rejects absolute input paths, parent traversal, symlink escape, and special files.
- All generated files remain beneath the trusted output root.

Security methodology

The implementation follows defense in depth: establish the caller-controlled trust boundary, bound every input-controlled expansion point, validate known simulation semantics early, preserve harmless extension data, and prevent generated artifacts or shared process state from becoming secondary attack surfaces.

Implementation themes

1. Centralized security helpers

- Added `security.jl` for path policy, size ceilings, safe filenames, CSV cells, finite-value checks, and overflow-safe dimension products.

- Kept rules explicit and reusable rather than duplicating policy at individual call sites.

2. Filesystem trust boundaries

- Separated input resolution from output resolution.

- Confined mode canonicalizes paths, verifies containment, blocks traversal and symlink components, and accepts only regular files.

- Local mode intentionally retains unrestricted trusted-user paths.

3. Bounded input parsing

- Configuration, profiles, weather, and geothermal data are size-limited before full processing.

- Replaced whole-file line loading with bounded iteration and added line-count, line-length, JSON-depth, and custom-data row ceilings.

4. Bounded computation and memory

- Added upper bounds for simulation steps, components, mesh cells, storage layers, stored output values, and related dimensions.

- Checked multiplied dimensions before allocation to prevent overflow and oversized arrays.

5. Parameter studies and optimizers

- Count Cartesian products before execution and reject excessive run counts.

- Avoid eager materialization; random sampling uses indexed combinations.

- Restrict optimizer kwargs and enforce evaluation/time ceilings after user options.

6. Graph-processing safety

- Added path-count and traversal-depth limits.

- Use visited-node handling and bounded traversal to prevent stack exhaustion or exponential work from pathological component graphs.

7. Compatible semantic validation

- **Known** ReSiE fields remain type-, range-, finiteness-, and structure-validated.

- Additional or unknown JSON fields are accepted for annotations, metadata, and forward-compatible extensions; they are ignored unless existing code explicitly uses them.

- Exact option matching replaces substring checks, and executable identifiers and algorithm names remain constrained.

8. Safe generated outputs

- All confined outputs use the output resolver; generated filenames are normalized and length-limited.

- CSV fields are quoted and formula-leading text is neutralized; generated HTML reprocessing is bounded.

Process integrity and compatibility

Shared state and logging

- Protect the global run registry with one process-wide lock for reads and mutations.

- Use task-scoped logging rather than replacing the process-global logger.

- Use UUIDv4 for externally visible run identifiers.

Error and status handling

- Public results no longer expose complete internal stack traces.

- Normalize log text to prevent control-character injection.

- Caught simulation failures propagate an unsuccessful job status rather than appearing successful.

Local compatibility

- Normal local calls require no mode argument.

- Configurations may remain under `examples/`; relative profile paths still resolve from the project root.

- Absolute and unrelated input/output locations remain supported in local mode.

Confined server entry point

- Added `load_and_run_confined(...)`, which always selects confinement and requires trusted roots.

- Added `run-confined <project> <input-root> <output-root>`.

- The server command exposes no local-mode or root-override flags.

Test infrastructure alignment

Changed	Method
runtests.jl	Run tests inside `with_logger`, close the logger in `finally`, and avoid replacing the process-global logger.
test_util.jl	Use UUIDv4, provide separate mock input/output path functions, and update the run registry while holding the shared lock.
Test group wrappers	No changes required; they only include lower-level test files.

Resulting security posture

The original engine behaved as a trusted local scientific application. The final version preserves that workflow while adding a distinct, fail-closed server interface and defensive limits inside the engine. Untrusted project data controls simulation content, but not the execution policy or trusted filesystem roots. Extra JSON metadata remains permitted and does not weaken validation of recognized simulation fields.

Validation and limitation

The final tree was compared with the original, patches were reapplied to clean copies, ZIP integrity and static delimiter checks were performed, and unsafe patterns were searched across the Julia source. A Julia runtime was not available in the working environment, so the modified project and full test suite were not compiled or executed.

Source basis: original archive `src(1).zip`, final archive `ReSiE_security_hardened_additional_json_source.zip`, complete v4 patch, and the updated test-infrastructure patch.